

Edge Detection for Object Boundary Extraction

A Comparison of the Sobel, Prewitt, LoG and Canny Operators in MATLAB

1. Aim

To detect the edges (object boundaries) in a digital image in MATLAB using four classical operators — **Sobel**, **Prewitt**, **LoG (Marr-Hildreth)** and **Canny** — and to compare their results using the number of edge pixels detected and the quality of the edges.

2. Introduction

An **edge** is a place in an image where the brightness changes sharply, which usually marks the **boundary of an object**. Finding these boundaries is one of the most important steps in machine vision, because an object can be recognised, measured or counted once its outline is known.

Edge detection reduces a full image to a thin set of boundary lines, throwing away the smooth interior and keeping only the useful structure. It is a core topic of **Unit 4 (Image Segmentation)** and is the first step in most object-recognition and inspection systems.

Edge detection is used everywhere in **robotics and automation** — lane detection in self-driving cars, part inspection on a production line, and shape measurement — so this work links directly to **SDG 9 (Industry, Innovation and Infrastructure)**.

3. Theory of the Operators

3.1 Edges and the Image Gradient

An edge is found by looking at how fast the intensity changes. The **gradient** measures this change in the x and y directions (G_x , G_y). Its **magnitude** tells how strong the edge is and its **direction** tells which way the edge runs:

$$|G| = \sqrt{G_x^2 + G_y^2} \quad \theta = \tan^{-1}(G_y / G_x)$$

A pixel is marked as an edge where this magnitude is large (above a threshold). **First-derivative** methods (Sobel, Prewitt, Canny) use the gradient directly; **second-derivative** methods (LoG) look for where the second derivative crosses zero.

3.2 Sobel and Prewitt (First-Derivative Masks)

Both slide a pair of 3×3 masks over the image to estimate G_x and G_y . **Sobel** gives the centre row/column a higher weight (2), which smooths noise a little, while **Prewitt** uses equal weights. The masks for the x-direction are:

Sobel G_x	Prewitt G_x
-1 0 +1	-1 0 +1
-2 0 +2	-1 0 +1
-1 0 +1	-1 0 +1

(G_y is the same mask rotated by 90 degrees)

They are simple and fast and give a strong response, but the edges they produce are **thick** and they are somewhat sensitive to noise.

3.3 LoG — Laplacian of Gaussian (Marr-Hildreth)

This is a **second-derivative** method. The image is first smoothed with a Gaussian (to reduce noise), then the **Laplacian** is applied; edges are the **zero-crossings** of the result (points where it changes sign). It produces thin, closed contours and can locate edges accurately, but it can miss weak edges and its result depends on the chosen Gaussian width (σ).

3.4 Canny (Optimal Multi-Stage Detector)

Canny is the most advanced operator and works in four stages: (1) **Gaussian smoothing** to remove noise; (2) find the **gradient** magnitude and direction; (3) **non-maximum suppression** to thin the edges to one pixel wide; and (4) **double thresholding with hysteresis**, which keeps strong edges and only those weak edges that are connected to strong ones. This gives **thin, continuous and well-located** edges with good noise resistance, which is why Canny is the most widely used edge detector.

4. Methodology

Implemented in MATLAB using the Image Processing Toolbox, mainly the built-in edge function. The steps are:

1. Read the input image and convert it to grayscale.
2. Apply the four operators with edge: sobel, prewitt, log and canny.
3. Display the original image and the four edge maps side by side.
4. Count the edge pixels in each result and find the edge density (percentage of white pixels).
5. Compare the operators on edge count, thickness, continuity and noise.

Data flow: Input image → Grayscale → Four edge operators → Display edge maps → Count edge pixels → Comparison.

5. MATLAB Code

The four operator names are kept in a list so a single loop applies edge, displays the result and counts the edge pixels in the same way for each. Keep **test.png** in the same folder as the script.

```
%% Edge Detection for Object Boundary Extraction
% Operators : Sobel, Prewitt, LoG (Marr-Hildreth), Canny
% Input      : test.png (any grayscale image)

clc; clear; close all;

% STEP 1 - Read the image and make sure it is grayscale
I = imread('test.png');
if size(I,3) == 3 % a colour image has 3 channels (R,G,B)
    I = rgb2gray(I);
end

% STEP 2 - Apply the four operators ('edge' returns a binary edge map)
methods = {'sobel','prewitt','log','canny'};
labels = {'Sobel','Prewitt','LoG','Canny'};
E = cell(1,4);
for k = 1:4
    E{k} = edge(I, methods{k});
end

% STEP 3 - Show the original and the four edge maps
```

```

figure('Name','Edge Detection');
subplot(2,3,1); imshow(I); title('Original');
for k = 1:4
    subplot(2,3,k+1); imshow(E{k}); title(labels{k});
end

% STEP 4 - Quality measure: number of edge pixels and edge density (%)
total = numel(I);
fprintf('%-10s %12s %10s\n','Operator','EdgePixels','Density(%)');
fprintf('%s\n', repmat('-',1,34));
for k = 1:4
    n = nnz(E{k}); % nnz = number of non-zero (edge) pixels
    fprintf('%-10s %12d %9.2f\n', labels{k}, n, 100*n/total);
end

```

6. Test Image

Figure 1 shows the input image and its gradient magnitude. The gradient is bright exactly along the object boundaries (the man, the camera and the tripod) and dark over the smooth sky and grass — these bright ridges are what the edge operators turn into thin lines.

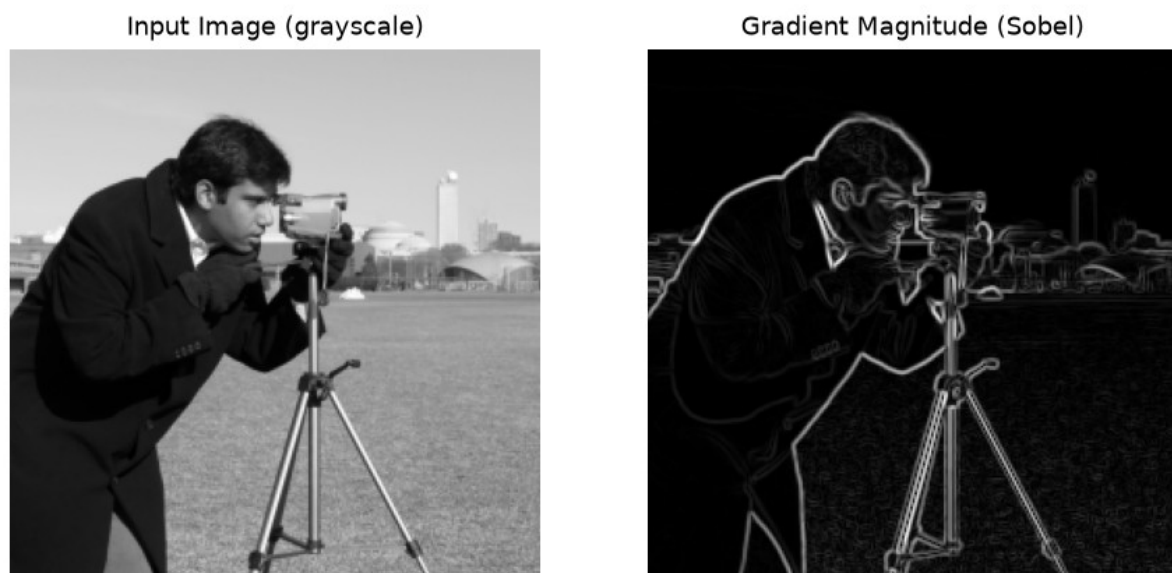


Figure 1: Input image (left) and its gradient magnitude (right).

7. Results and Output Images

Figure 2 shows the edge map from each operator. Sobel and Prewitt give very similar, slightly thick outlines; LoG gives thin contours but drops many weak edges; Canny gives the thinnest, most continuous edges and recovers the most real detail of the object. Figure 3 shows the Canny edges drawn on top of the original image, which is the final object-boundary result.

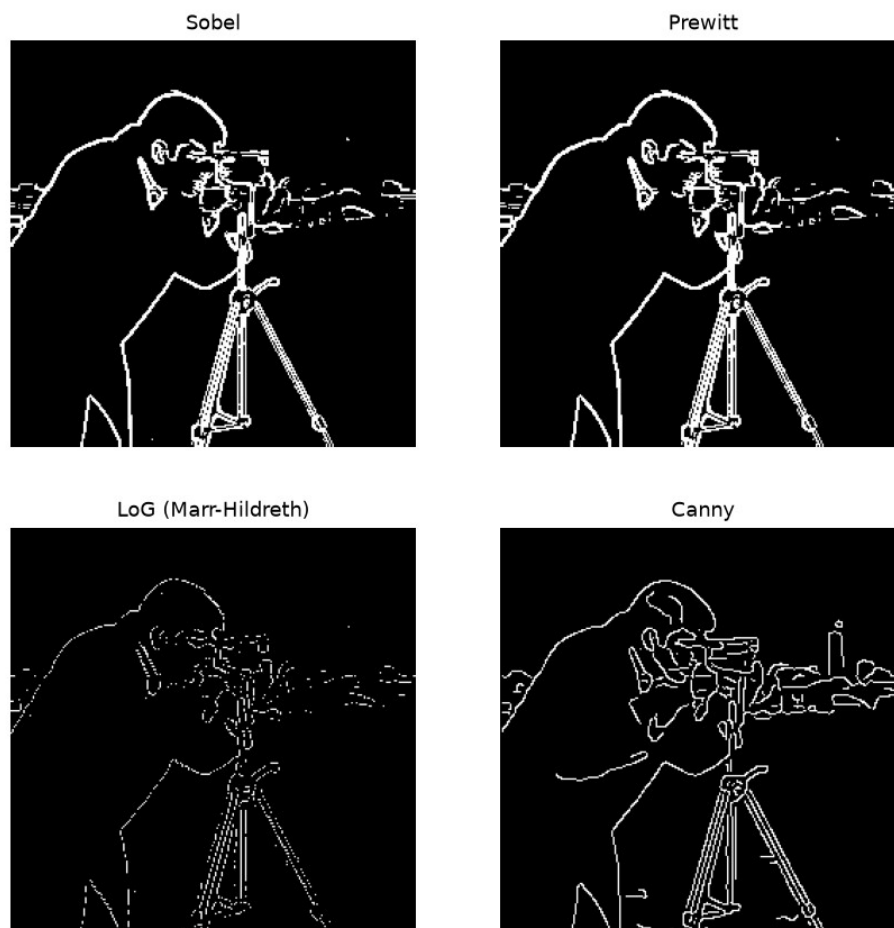


Figure 2: Edge maps produced by the four operators.

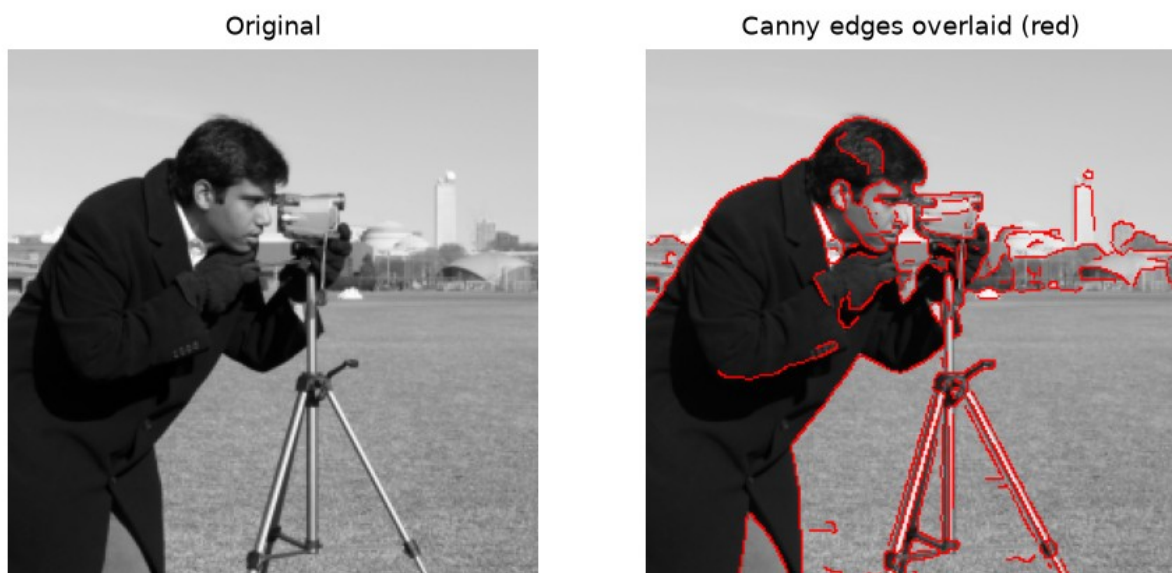


Figure 3: Canny edges (red) overlaid on the original image — the extracted object boundary.

8. Quantitative Results

The table lists how many pixels each operator marked as an edge, for the 256×256 (65 536-pixel) test image. "Edge density" is the percentage of the image that became edge pixels.

Operator	Edge Pixels	Edge Density (%)	Edge Appearance
Sobel	4 656	7.10	Thick, strong
Prewitt	4 834	7.38	Thick, strong
LoG (Marr-Hildreth)	1 350	2.06	Thin, some gaps
Canny	3 351	5.11	Thin, continuous

9. Result Analysis

- **Sobel and Prewitt** marked the most pixels as edges (about 7%), but the edges are **thick** and a thick edge is less precise about exactly where the boundary lies. Their results are almost identical, with Sobel slightly cleaner because of its extra smoothing.
- **LoG** gave the fewest edge pixels (about 2%). Its zero-crossing contours are thin and neat, but it **missed several weak edges** and is sensitive to the chosen σ and to noise.
- **Canny** gave a moderate number of edge pixels (about 5%) but the edges are **one pixel thin, continuous and well-placed**, and it recovered the most genuine object detail (the camera and the face) without filling the image with noise.

A good edge map is not the one with the **most** edge pixels but the one whose edges are **thin, continuous and in the right place**. By this standard **Canny is the best operator**, because its hysteresis thresholding and non-maximum suppression give clean, connected boundaries and good noise handling.

10. Advantages and Limitations

Operator	Advantage	Limitation
Sobel / Prewitt	Simple, fast, strong response	Thick edges; sensitive to noise
LoG (Marr-Hildreth)	Thin, closed contours; sub-pixel location	Misses weak edges; σ - and noise-sensitive
Canny	Thin, continuous, accurate; good with noise	Slower; needs two thresholds and σ

11. Applications

Edge detection is used in **object recognition and tracking**, **lane and obstacle detection** in self-driving cars and mobile robots, **industrial inspection** (checking part shape and defects), **medical imaging** (outlining organs and tumours), fingerprint and number-plate recognition, and satellite/remote-sensing image analysis.

12. Conclusion

Four classical edge-detection operators were implemented in MATLAB and tested on a grayscale image. Sobel and Prewitt produced strong but thick edges, LoG produced thin contours but missed weak edges, and **Canny produced the thinnest, most continuous and most accurate object boundaries** with good noise resistance. **Canny is therefore**

the most suitable operator for object boundary extraction. The project meets CO-1 and CO-4 and supports SDG 9 (Industry, Innovation and Infrastructure).

13. Course Outcome and SDG Mapping

CO / SDG	How this project addresses it
CO-1	Implements image-processing algorithms (edge operators) in MATLAB.
CO-4	Applies image segmentation and the mathematics of the image gradient and Laplacian.
SDG 9	Supports industry and innovation — edge detection underpins automation, robotics and inspection.

14. References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Addison-Wesley.
- [2] J. Canny, "A Computational Approach to Edge Detection," *IEEE Trans. PAMI*, 1986.
- [3] D. Marr and E. Hildreth, "Theory of Edge Detection," *Proc. Royal Society of London*, 1980.
- [4] MATLAB Documentation — Image Processing Toolbox: `edge`, `rgb2gray`, `imshow`.